
INDEPENDENT SECURITY AUDIT · v0.6.2

Bitcoin Easy Signer

A full-stack review of the application's own code, the open-source components it inherits, and the pipeline that builds and signs each release. No path was found that could move funds without the operator's approval, expose key material, or alter a transaction unseen.

Prepared by Z.ai (GLM-5.3, via ZCode) · Lead auditor, three specialist agents, independent verification · 2 October 2026

Contents

Document Control

Executive Summary

Findings in Brief

What Was Verified Intact

Scope and Method

Coverage and Limitations

Next Audit Cycle: the Color Team Panel

Appendix A — Findings Ledger

Appendix B — Reproduction and Sources

1

2

3

4

4

5

5

6

7

Document Control

Field	Detail
Report title	Independent Security Audit — Bitcoin Easy Signer v0.6.2
Report identifier	DOC ZAI-BESA-062 · designed edition of the report published at releases/AUDIT-ZAI-0.6.2.md
Subject	Bitcoin Easy Signer v0.6.2 — a macOS application that guides a non-technical operator (executor, trustee, attorney) through a payment from an existing 2-of-3 multisig Bitcoin wallet
Version examined	Tag v0.6.2 · commit 7b4db8548df2132ee031153725c45391c0f3103f · published 2 October 2026
Auditor	Z.ai (GLM-5.3, via ZCode) — lead auditor, three specialist sub-agents (cryptography; hardware and transport; supply chain), and an independent verification agent that re-derived every load-bearing claim before publication
Method	Full read of the application's own code; line-by-line review of the inherited libraries it depends on; inspection of the build, signing, notarization and release pipeline; independent re-verification of the published artifacts
Classification	Public — released to the project repository
Date of audit	2 October 2026

Findings in this report carry stable identifiers (BESA-01 through BESA-25). Remediation for each finding is scheduled for the next release cycle; the project's release records track each identifier to its fix.

Executive Summary

Bitcoin Easy Signer never holds the wallet's keys: it has no code path that reads a seed phrase, a private key, or a device PIN — a property verified by tracing every input the software accepts, not by taking documentation at its word. Every payment must be approved on the screens of the number of hardware signing devices the wallet's definition file requires (two of three for the wallets this software is built for), and the application cryptographically checks that what the devices signed is exactly what the operator reviewed before anything is sent.

25

FINDINGS, IN TOTAL

0

CRITICAL FINDINGS

249TESTS RE-RUN AND
PASSING**3**ARTIFACTS RE-HASHED
AND MATCHED

The four questions that matter

Question	Verdict
Could this software send bitcoin somewhere the operator did not approve?	No — no such path was found. A payment leaves the machine only after the operator confirms the exact destination, amount, fee and change; the confirmed transaction ID is checked against the one built; every output is re-compared field-by-field against that review; the coins spent are re-checked as unspent against two independent public sources on mainnet; and the network's answer is checked against the reviewed ID.
Could this software expose a seed phrase, private key, or device PIN?	No — no such path was found. The wallet-file parser rejects private key material outright; a Blockstack Jade's PIN is typed on the device and never reaches the computer in any form; the only disk writes are an unsigned transaction file, a fixed-vocabulary troubleshooting report, and explorer web addresses.
Could a remote party, a dependency, or a local process alter a transaction without the operator seeing it?	No — not on any path this software actually executes. Two defects of exactly this class exist in the inherited embit library; both were traced to dead ends (the application does not use the affected code, and the affected values cannot occur in transactions it builds). The audit's hostile-input tests — tampered device responses, malicious explorer data, forged signatures — were all refused.
What should be fixed first?	One gap in the build pipeline: a single support package is installed in the release build without the hash verification every other dependency has, in the same job that holds the signing certificate. The published v0.6.2 artifact is unaffected and verifies cleanly; the fix is a one-line lock-file change.

This report is evidence about one version, examined on one date. It is not a guarantee, and it does not make any future payment safe. Its coverage limits are stated in a later section and should be read alongside these verdicts.

Findings in Brief

No finding provides a way to steal bitcoin from a user of v0.6.2. The findings below are real and are reported at their true severity; most concern the machinery that builds future releases, or latent defects in inherited libraries that this application provably does not reach.

ID	Severity	Area	Finding
BESA-01	High	Release pipeline	One support package (pyyaml) installed without hash verification in the CI job holding the signing certificate. Affects future builds only; the published artifact verifies intact.
BESA-02	High*	Library (embit)	The library's stock transaction finalizer performs no signature verification. The application never uses it — it has its own finalizer that verifies every signature cryptographically.
BESA-03	Medium	Release pipeline	Fallback fetches the USB library via Homebrew — unverified code — on the runner that holds signing secrets.
BESA-04	Medium	Release pipeline	Signing credentials remain available to the whole build job after unlock, widening the blast radius of any unpinned execution point.
BESA-05	Medium	Release pipeline	The publish step never re-checks file digests against the published checksum list, and the immutability guard tests release existence rather than tag existence.
BESA-06	Medium	USB library	The digest-pinned libusb guarantee cannot be checked on the shipped product: bundling re-signs the file, and the bill of materials records the pre-signing digest.
BESA-07	Medium	USB library	On Macs with Homebrew, the device code loads Homebrew's copy of the USB library instead of the bundled one, because of a file-name mismatch.
BESA-08	Medium	Devices	Signer identity is verified when devices are scanned, not re-checked at the instant of signing; a swapped device still cannot pass signature verification, so the bound is cryptographic rather than procedural.

* Library-level severity. The defect exists in the inherited open-source library but was verified unreachable in, or contained by, this application's code.

The remaining seventeen findings are lower-severity control and availability gaps — a missing interpreter entry in the bill of materials, a stale pre-publication stamp in the source archive, fail-closed parsing quirks in the embit library (fixed in its newer releases; an upgrade is scheduled), and informational observations. The complete ledger of all twenty-five findings appears in Appendix A.

What Was Verified Intact

Every claim below was independently re-derived during the audit, not taken from project documentation.

Control	Verification
Published artifacts	SHA-256 digests of the application disk image, source archive and software bill of materials recomputed and matched against the published checksum list.
Code signing and notarization	Developer ID signature (Bitseeker LLC, team B8G5L7M8TB), hardened runtime, strict deep verification, Apple notarization ticket and stapling all validate on the downloaded file; Gatekeeper accepts it.
Test suites	240 automated application tests and 9 interface tests re-run at the audited source revision; all pass.
Build automation	All four third-party build actions pinned to exact commits, each resolving to the claimed release; publishing is manual-dispatch only, with no push or pull-request path to a release.
Dependency integrity	Every Python package that ships in the application is installed under hash-lock, end to end (the single exception is BESA-01, which does not ship).
Prior-issue records	The five issues recorded by the project before this audit are each accurately described in its documents; the fix scheduled for this version was verified present in code.

Scope and Method

The audit covered three layers. First, the application's own code: every module was read in full, including the entire operator interface, the local server that powers it, the transaction construction and verification engine, and the device-communication layer. Second, the inherited open-source components the application depends on — the embit Bitcoin library (validated line-by-line and against Bitcoin's official test vectors: BIP-32, BIP-143 across all six signature-hash modes, and BIP-173), Bitcoin Core's hardware-wallet interface (HWI 3.2.0), the bundled USB library, and the windowing and packaging stack — including inspection of the shipped binary's embedded contents. Third, the supply chain: the build workflow, signing and notarization pipeline, lock files, and the published artifacts themselves.

The work was performed by a lead auditor with three specialist sub-agents (cryptography; hardware and transport; supply chain), followed by an independent verification agent that attempted to disprove each load-bearing claim before publication. All ten claims it re-derived held. The audit was conducted read-only: no repository files were modified, no transaction was created or broadcast on any network, and no real wallet material was handled at any time.

Coverage and Limitations

What this audit did not do — stated plainly:

- No physical hardware signing device was attached. Device-screen behavior rests on the project's recorded hardware evidence, and the counterfeit-device analysis is code-level.
- No transaction was created, signed, or broadcast on any network; no live explorer was contacted by the auditor.
- The internals of GitHub's and Apple's infrastructure were not (and cannot be) audited; the published artifact's validity rests on the checks that were performed.
- The binary cannot be reproduced byte-for-byte from source (a property of the packaging tool, documented by the project itself); the tie between the published application and the audited source commit is the recorded build evidence plus those checks.
- “No finding” means “none found within this coverage,” not “none exist.”

Next Audit Cycle: the Color Team Panel

The follow-up audit (v0.6.3) will be performed by a named panel of specialist agents, each with one lens and a charter locked in writing before the build is examined. Red and blue are established security-industry terms; the remaining lenses and the referee are this format's extensions. The grade definitions are likewise locked in advance: green requires zero open critical or high findings, every prior finding verified fixed, and all stated defenses held and regression-tested. The overall grade is the floor of the panel, never the average.

	Agent	Role	Charter
	Red	The attacker	Tries to steal the money, the keys, or the operator's decision — by any path. Reads no prior conclusions, so it inherits no one's blind spots.
	Blue	The defender	Proves every stated protection actually holds and is pinned by a test that fails if anyone breaks it.
	Orange	The cryptographer	Checks the mathematics and money semantics: signatures, derivation, transaction building, and what changed in the libraries since this audit.
	Copper	The hardware specialist	Checks everything between the application and the physical signing devices: USB transport, device identity, firmware bridges.
	Amber	The supply-chain inspector	Checks how the artifact is born: every dependency, build step, signature and download in the chain.
	White	The referee	Sees everything, re-verifies every load-bearing claim personally, and gates what gets published.

Appendix A — Findings Ledger

All twenty-five findings. Severity markers: * library-level severity, verified unreachable or contained in this application. Every finding has a scheduled remedy in the next release cycle; the project's release records map each identifier to its fix.

ID	Sev.	Area	Finding
BESA-01	High	Pipeline	pyyaml installed without hash verification beside signing credentials
BESA-02	High*	Library	embit stock finalizer verifies no signatures; unused by this app
BESA-03	Medium	Pipeline	Homebrew fallback executes unverified code on the signing runner
BESA-04	Medium	Pipeline	Signing secrets exported job-wide before untrusted steps
BESA-05	Medium	Pipeline	No digest re-check at publish; guard checks releases, not tags
BESA-06	Medium	USB library	Digest pin unverifiable on shipped bytes; SBOM records pre-signing value
BESA-07	Medium	USB library	usb1 loader prefers a Homebrew copy over the bundled library
BESA-08	Medium	Devices	Signer identity checked at scan, not re-checked at signing
BESA-09	Medium*	Library	embit truncated-input parsing fails open; contained downstream
BESA-10	Medium*	Library	embit rewrites zero-valued transaction fields; unreachable constants here
BESA-11	Low	USB library	Without any system libusb the device bridge fails closed (availability)
BESA-12	Low	USB library	Preload fails open if bundled dylib missing; extraction dir writable
BESA-13	Low*	Library	Address decoding lacks a network parameter; app round-trip check closes it
BESA-14	Low*	Library	Descriptor checksum suffix ignored by library; app validates it
BESA-15	Low*	Library	PSBT input-map length handling weaker than BIP-174; outer checks reject
BESA-16	Low*	Library	Miniscript compile denial-of-service family; app allowlist blocks triggers
BESA-17	Low	Library	embit 0.8.0 prefers bundled prebuilt crypto binaries; upgrade recommended
BESA-18	Low	SBOM	Shipped CPython runtime absent from bill of materials; pip listed but not shipped
BESA-19	Low*	Library	Library exceptions escape one explorer-data catch (fail-closed, wrong message)
BESA-20	Low	Artifacts	Application-to-source binding rests on build-runner trust; honestly documented
BESA-21	Low	Artifacts	Source archive ships stale pre-publication status stamps
BESA-22	Info	Product	No auto-update mechanism (verified); lightweight tags pose no weakness here
BESA-23	Info	Devices	Jade PIN never enters the host; pin-server relay target is device-chosen
BESA-24	Info	Ecosystem	No known HWI vulnerabilities found; project is in maintenance mode
BESA-25	Info	Library	Minor observations: fee arithmetic edge cases; trust store is a documented union

Appendix B — Reproduction and Sources

The markdown edition of this report, the audited source at the tagged commit, the published artifacts and their checksums, and the project's release evidence records are publicly available:

- Repository: github.com/cjtsh/bitcoin-easy-multisig-signer
- This report (markdown edition): [releases/AUDIT-ZAI-0.6.2.md](#)
- Audited revision: tag v0.6.2, commit 7b4db85 (full identifier in the Document Control table)
- Release artifacts: DMG, source archive, software bill of materials and SHA256SUMS, attached to the v0.6.2 release

Audit performed 2 October 2026 by Z.ai on the public repository and published artifacts only. No wallet material, addresses, or transaction identifiers appear in this report. An audit is not a certification of safety; it is dated evidence about one revision, and its coverage limits are stated above.